

Module Name

Inventory Ingestion & Normalization Engine (IINE)

1. Goal and Objective of the Module

This module serves as the entry point for all inventory and financial truth within **KDPS**. Its job is not limited to “processing invoices” or “uploading PT files.” It is responsible for converting **real-world, inconsistent, and delayed inputs** into a **clean, structured, and auditable dataset that downstream systems (especially accounting systems)** can trust.

Core Objective

Create a system that ensures:

- Every piece of inventory received by KDPS is:
 - **physically validated**
 - **financially traceable**
 - **semantically standardized**
- All vendor inputs (PT files, invoices, physical goods) are:
 - **normalized into a unified KDPS schema**
 - **transformed into TIR-compatible format**
 - **ready for accounting without manual rework**

*Within KDPS, **TIR** refers to the structured transaction format required by the accounting system (Tally/TIR software) to ingest purchase and inventory data. It is not a vendor file and not an industry standard. It is: “**The final, accounting-ready dataset that records inventory value, tax, and payable to vendors**”*

*This module’s output must always be: **100% compatible with TIR format**, without requiring manual rework*

What success looks like

- Zero dependency on Excel-based transformations
- Minimal manual classification work
- Real-time visibility of:
 - What was ordered
 - What was received
 - What is payable
- Consistent product taxonomy across:

- branded
 - non-branded inventory
 - Seamless flow into accounting with **no reformatting effort**
-

2. Current Workflow (As-Is State)

The current workflow is fragmented, manual, and highly dependent on human interpretation. It can be broken down into three parallel, but loosely connected, processes.

A. Physical Inventory Receipt (Warehouse / Store)

Trigger:

Inventory arrives at the warehouse or store.

Steps:

- Items are manually matched **against the invoice**
- Validation includes:
 - quantity check
 - item presence
 - damage identification
- Issues (shortage/damage) are:
 - noted manually
 - communicated to the brand via the account manager

Output:

- Informal confirmation to the head office:
 - "Received as expected" OR
 - "Discrepancy exists."
-

B. PT File Dependency (Branded Flow)

Reality:

- PT file may:
 - arrive with shipment
 - arrive later
 - require chasing by the account manager

Steps:

- The PT file is opened manually
- Columns differ per brand
- Data is interpreted manually
- Mappings are applied:
 - category
 - subcategory
 - type
 - fit

Pain points:

- No standard format
 - No automated mapping
 - Heavy reliance on experience
-

C. Non-Branded Flow (Manual Data Creation)**Reality:**

- No PT file exists
- An invoice is the only structured input
- Products may not have:
 - barcode
 - meaningful description

Steps:

- Human interprets product from:
 - invoice
 - physical inspection
- Creates:
 - product attributes
 - category mapping
 - pricing
 - tax
 - margin
- Generates:
 - internal PT file
 - sometimes barcode

Pain points:

- Subjective classification
 - Inconsistent SKU creation
 - High cognitive load
-

D. Transformation to Accounting Format

Steps:

- Data is manually reshaped into a TIR-compatible format
- Uploaded into the accounting system

Output:

- Inventory value reflected
 - Payable to the brand recorded
 - Approval → Payment initiated
-

3. Where Human Intervention Wastes Time

This system is currently **human-heavy at every stage**, which introduces inefficiency, inconsistency, and risk.

1. File Handling and Interpretation

- PT files:
 - different formats
 - messy headers
 - inconsistent columns
 - Requires:
 - manual cleaning
 - column identification
-

2. Semantic Mapping

- Brand → KDPS taxonomy mapping is:
 - undocumented
 - person-dependent
- Example:

- “Formal wear” → “Casual wear.”
 - “Chinos / Shorts / Boxers” → unified subcategories
-

3. Non-Branded Product Creation

- The entire process is manual:
 - product identification
 - categorization
 - pricing logic
 - No guardrails:
 - leads to duplication
 - inconsistent naming
-

4. Reconciliation Gaps

Currently missing structured tracking for:

- Invoice vs Physical Receipt
- Invoice vs PT file
- PT vs Final Accounting Entry

This creates:

- hidden discrepancies
 - delayed detection of errors
-

5. Dependency Bottlenecks

- Accounting depends on PT transformation
 - PT files may be delayed
 - Work gets blocked waiting for:
 - vendor
 - account manager
-

4. System Gaps and Improvement Opportunities

This is where the current system is structurally weak.

A. No Unified Data Model

- Every brand operates differently
 - No internal canonical schema
 - Leads to:
 - reporting inconsistency
 - poor analytics
-

B. No Decoupling of Flows

Three critical processes are tightly coupled:

- Physical receipt
- PT processing
- Accounting entry

They should instead be:

- independent
 - asynchronously processed
-

C. No Rule Engine

Mapping logic is:

- implicit
- manual
- non-versioned

Needed:

- configurable rules
 - brand-specific logic
 - override capability
-

D. No SKU Identity Control

Especially in non-branded:

- same product → multiple SKUs
- no reuse mechanism

Needed:

- centralized SKU registry
 - similarity detection
-

E. No GRN-Centric System

GRN (Goods Receipt Note) is not formalized as a system object.

Needed:

- GRN as **primary anchor**
 - tied to:
 - invoice
 - receipt
 - discrepancies
-

F. No Audit Trail

- Who mapped what?
- Why was the classification changed?

Currently:

→ not traceable

5. Proposed Product Design (To-Be State)

This module becomes a **central operating layer between physical inventory and accounting systems.**

A. User Interfaces

1. Warehouse / Store Mobile & Web App

Used by:

- warehouse staff
- store managers

Capabilities:

- Upload invoice (PDF/image)
- Create GRN:
 - Confirm received items
 - mark shortages/damages
- Scan barcode (if available)
- Manual item entry (fallback)

Key rule:

→ No downstream processing without invoice upload

2. Head Office Dashboard (Web)

Used by:

- account managers
- operations team
- finance team

Capabilities:

a. PT File Management

- Upload PT file
- Auto-parse
- Map columns (once per brand/file type)

b. Transformation Engine

- Apply:
 - brand-specific mapping rules
 - category normalization
- Preview output

c. Non-Branded Product Creation

- Create new product:
 - category
 - attributes
 - pricing

- Suggest similar SKUs
- Generate barcode

d. Reconciliation Panel

- View:
 - Invoice vs GRN
 - Invoice vs PT
 - Highlight mismatches
-

B. System Components

1. Ingestion Engine

- Handles:
 - invoice upload
 - PT file upload
 - Detects:
 - headers
 - structure
-

2. GRN Engine

- Creates:
 - receipt record
 - Tracks:
 - shortages
 - damages
-

3. Normalization Engine

For Branded:

- PT → standardized schema

For Non-Branded:

- manual input → structured dataset
-

4. Rule Engine

- Brand-specific mappings:
 - category
 - subcategory
 - Version-controlled
 - Editable without code
-

5. SKU & Barcode Engine

- Unique SKU creation
 - Barcode generation
 - Duplicate detection
-

6. Reconciliation Engine

Tracks:

- Invoice vs GRN
 - Invoice vs PT
 - Final approval status
-

7. TIR Transformation Layer

- Converts data into:
 - strict accounting format
-

C. Workflow (Future State)

Step 1: Invoice Upload

- Mandatory
 - Triggers system entry
-

Step 2: GRN Creation

- Physical validation
 - Independent of PT
-

Step 3: PT Processing

Branded:

- Upload PT
- Auto-map
- Normalize

Non-Branded:

- Create product entries
 - Generate PT internally
-

Step 4: Reconciliation

- System flags:
 - mismatches
 - missing data
-

Step 5: Accounting Export

- TIR-ready file generated
 - No manual reshaping
-

Step 6: Approval & Payment

- Outside module scope
- Enabled by clean data

Module Name

Payment Authorization & Settlement Control System (PASCS)

Note on Context

This module sits **after** the Inventory Ingestion & Normalization Engine (IINE).

IINE ensures:

- The invoice is validated
- GRN is confirmed
- PT data is normalized
- TIR (accounting-ready dataset) is generated

This module ensures:

→ **money is released only when all upstream truths are aligned and approved**

1. Goal and Objective of the Module

This module governs **how and when KDPS pays its vendors (brands)**.

It is not a simple “payment tracker.”

It is a **financial control system** that ensures:

- payments are:
 - justified
 - approved
 - traceable
 - No payment is made:
 - without reconciliation
 - without authority
 - without an audit trail
-

Core Objective

Create a structured system where:

- every payable transaction:
 - originates from validated operational data
 - passes through a defined approval hierarchy
 - is executed with full traceability
 - every payment:
 - is linked to:
 - invoice
 - GRN
 - PT
 - TIR
 - can be audited at any time
-

What success looks like

- No informal or ad-hoc payments
 - No payment without full reconciliation
 - Clear approval hierarchy enforced
 - Real-time visibility into:
 - pending payments
 - approved payments
 - completed payments
 - Seamless link between:
 - operational data
 - accounting records
 - bank transactions
-

2. Current Workflow (As-Is)

The current payment process is partially structured but lacks system-driven control.

A. Trigger for Payment

Payment is considered when:

- Inventory is received
- The invoice is validated
- The PT file is processed
- Data is ready for accounting

However:

- This validation is not system-enforced
 - It depends on human confirmation
-

B. Approval Process

- The payment request is sent to a senior person
- Approval is given manually
- No structured workflow system

Issues:

- No defined hierarchy in the system
 - No tracking of:
 - who approved
 - when
 - based on what
-

C. Payment Execution

- Someone manually logs into the bank
- Makes payment
- Marks internally:
 - “payment done”

Issues:

- No structured linkage to:
 - invoice
 - TIR
 - No standardized capture of:
 - transaction ID
 - payment mode
-

D. Post-Payment Tracking

- Later, in accounting:
 - bank statements (PDF) are uploaded
 - Transactions are matched manually

Issues:

- Matching is reactive, not proactive
 - No system-level linkage between:
 - payment intent
 - payment execution
 - bank confirmation
-

3. Where the Current System Breaks

1. Weak Control Before Payment

- No enforced validation:
 - Invoice vs GRN
 - Invoice vs PT
 - Risk:
 - paying for incorrect quantities
 - paying before the resolution of discrepancies
-

2. No Formal Approval Hierarchy

- Approval depends on:
 - informal processes
 - human memory

No system ensures:

- correct authority level
 - escalation logic
-

3. No Payment Traceability

- Payment is not tightly linked to:
 - transaction data
- Missing:
 - transaction ID capture
 - structured audit trail

4. Disconnected from Bank Reality

- Payment marked “done” internally
- Bank confirmation happens later

This creates:

- reconciliation gaps
 - delayed error detection
-

5. No Closed-Loop System

Currently:

Approval → Payment → Separate bank reconciliation

What's missing:

→ **single lifecycle tracking of payment**

4. System Gaps and Improvement Areas

A. No Unified Payment Object

There is no single entity that tracks:

- payable amount
 - approval status
 - payment status
 - bank confirmation
-

B. No Rule-Based Approval System

Needed:

- role-based authority
- thresholds (amount-based approvals)

- multi-level approvals
-

C. No Pre-Payment Validation Gate

System must enforce:

- reconciliation complete
 - discrepancies resolved
-

D. No Transaction-Level Traceability

Every payment must store:

- bank reference ID
 - payment timestamp
 - payment mode
-

E. No Integration Between Systems

- Operations
- Accounting
- Banking

Currently disconnected.

5. Proposed Product Design (To-Be)

This module becomes a **controlled payment lifecycle system**.

A. Core Concept: Payment Lifecycle

Each payment moves through:

Eligible → Requested → Approved → Executed → Confirmed → Reconciled

B. User Roles & Hierarchy

System must define:

1. Requestor

- initiates payment request

2. Approver(s)

- one or multiple levels
- based on:
 - amount
 - vendor
 - risk level

3. Executor

- performs payment

4. Finance Controller (optional)

- final oversight
-

C. Payment Workflow

Step 1: Payment Eligibility

System checks:

- Invoice validated
- GRN completed
- PT processed
- TIR generated

Only then:

→ payment becomes “eligible.”

Step 2: Payment Request Creation

- System creates a payment request:

- vendor
 - amount
 - invoice references
-

Step 3: Approval Workflow

- Routed based on hierarchy
- Each approver:
 - approves/rejects
 - adds comments

System records:

- who approved
 - timestamp
 - decision
-

Step 4: Payment Execution

- The executor makes payment via bank
 - System requires entry of:
 - transaction ID
 - payment mode
 - bank account used
-

Step 5: Payment Confirmation

- Status updated to:
→ "Executed"
-

Step 6: Bank Reconciliation (Future Accounting Module Link)

Later:

- Bank statement (PDF) uploaded
- System matches:
 - transaction ID
 - amount

- date

Status updated to:

→ “Reconciled”

D. User Interfaces

1. Payment Dashboard (Web)

Shows:

- Pending approvals
 - Approved payments
 - Executed payments
 - Reconciliation status
-

2. Approval Interface (Web/Mobile)

- View full context:
 - invoice
 - GRN
 - PT summary
 - Approve/reject
-

3. Execution Interface (Web)

- Enter:
 - transaction ID
 - payment details
-

4. Reconciliation Interface

- Upload the bank statement
 - Match transactions
-

E. System Components

1. Payment Engine

- manages lifecycle
-

2. Approval Engine

- hierarchy
 - routing
 - thresholds
-

3. Validation Engine

- ensures eligibility
-

4. Transaction Capture Layer

- stores payment details
-

5. Reconciliation Engine

- matches bank records
-

6. Future Workflow (End-to-End)

Inventory Validated (IINE)



TIR Generated



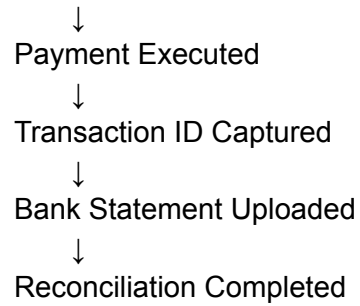
Payment Eligible



Payment Request Created



Approval Workflow



Final Positioning

This module is:

A financial control layer that ensures every rupee paid by KDPS is justified, approved, executed, and traceable

It connects:

- operations
- accounting
- banking

into a single, auditable payment lifecycle.

Module Name

Store Inventory Movement, Adjustment & Sell-Through Control System (SIMAS)

1. Goal and Objective of the Module

This module governs what happens **after inventory becomes valid store inventory**.

The earlier module handled inbound truth:

- what was invoiced
- what was received
- what was normalized
- what was accepted into the system

This module handles **operational truth at store level**:

- what stock is currently in the store
- how it moves
- how it gets reduced
- how it is transferred
- how it is returned
- how it is billed under special commercial scenarios
- how the system continues to reflect the difference between **physical stock, system stock, and commercially billed stock**

That distinction matters a lot. In a normal retail setup, people often simplify inventory as:

$$\text{Opening stock} + \text{inward stock} - \text{sales} = \text{closing stock}$$

That is too simplistic for KDPS's MBO environment.

Here, store inventory does not reduce only because of customer sales. It also changes because of:

- store-to-store transfers
- defect identification and replacement
- return rights granted by brands
- end-of-season commercial billing events where stock is billed but may still remain physically inside the store

So the objective of this module is not just stock counting. Its real purpose is to build a **controlled, auditable store inventory ledger** that reflects every legitimate reason why stock reduces, shifts, gets reclassified, or exits commercial ownership.

Core objective

Create a store operations system that ensures:

- every accepted inward is reflected correctly in store inventory
- every inventory reduction event is captured through a defined workflow
- every movement between stores is digitally authorized and acknowledged
- every defect, return, and seasonal commercial adjustment is traceable
- system stock, physical stock, and commercially billed stock are not mixed up
- store-level inventory remains accurate enough for:
 - sales
 - replenishment
 - transfer decisions
 - commercial settlement
 - loss control
 - end-of-season review

What success looks like

A successful version of this module would mean:

- No handwritten transfer memos
- No ambiguity on whether stock is:
 - physically present
 - system-owned
 - billed but retained
- Real-time visibility into store stock by SKU
- Clean transfer workflows between stores
- Separate treatment of normal sales vs internal billing events
- Clear handling of defective stock and returns
- Better seasonal decisions on:
 - transfer
 - return
 - EOSS billing
 - residual stock management

2. What This Module Covers

This module begins **only after inventory has been accepted and added to store inventory**.

That accepted stock may have reached the store through two possible routes:

Route 1: Direct inward to store

Brand sends inventory directly to the store, the store validates receipt, and accepted inventory gets added to store stock.

Route 2: Warehouse to store transfer

Inventory first reaches warehouse, is accepted there, and is later sent to the store. Once the store confirms receipt and acceptance, it becomes part of store inventory.

In either case, once stock is accepted into a store, this module becomes responsible for every subsequent change affecting that stock.

3. Current Business Reality

Today, store inventory changes through multiple operational events. Some are formal, some semi-formal, and some are still dependent on manual practices.

The practical business problem is that store stock is not a single static number. It is constantly moving between several commercial states.

A store starts with opening stock. Then new stock is added. After that, stock can reduce in several different ways. If the system treats all reductions as “sales,” the numbers will look clean on paper but will become misleading in reality.

That is exactly why this module is needed.

4. Store Inventory Logic

The system should treat store inventory as a continuously updated ledger:

Base logic

Store Available Stock = Opening Accepted Stock + New Accepted Inward - All Valid Reductions - All Pending Outbound Not Yet Accepted

But this formula alone is not enough. The system must classify reductions into different types because each type has different commercial meaning and different operational behavior.

For KDPS, the main reduction and movement types described so far are:

1. Customer Sale through POS

2. Internal Transfer to another store
3. Defective stock identified after inward
4. Brand-approved return / seasonal return
5. EOSS billed stock where inventory is commercially billed but physically retained in store

These are not interchangeable. Each one must have its own workflow, status model, and audit trail.

5. Current Workflow and Where Human Effort is Wasted

At present, some of these activities are happening through operational habit rather than a disciplined digital workflow.

A. Inventory inward acknowledgement

When stock reaches store from warehouse or directly from brand, a receipt is shared and store inventory is updated. But the process can vary depending on source and document type.

B. Internal transfers

These currently happen using handwritten memos. That means:

- no system visibility before dispatch
- no structured acceptance at destination
- weak accountability for missing or damaged items in transit
- no clean stock reservation logic during movement

C. Defect reporting

Defects may be found later, after stock has already been accepted. Raising concerns to the brand and tracking replacement is often operationally messy.

D. Seasonal return and EOSS handling

This is the biggest complexity area. Some portion of inventory can be returned to the brand under contract. Some gets sold under EOSS. Some gets commercially billed by the store itself even though it remains physically inside the store. If these are not recorded separately, the system will confuse:

- sale
- return
- self-billed retained inventory
- actual physical stock

E. Lack of dual stock visibility

The biggest gap is that the business needs to know not just what is in the store physically, but what is:

- available for normal sale
- reserved for transfer
- defective and quarantined
- approved for return
- commercially billed but physically retained

Without this, stock decisions will remain weak.

6. Key Improvement Principle

This module should not be built as a simple stock deduction tool.

It should be built as a **store-level inventory state management system**.

That means every SKU at a store can exist in one of several states, such as:

- Available for Sale
- Pending Acceptance
- Pending Transfer Out
- In Transit
- Accepted at Destination
- Defective / Quarantined
- Return Approved
- Returned to Brand
- Sold to Customer
- EOSS Billed and Retained
- Liquidated / Disposed (future extension)

This state-based model will make the module much more accurate than a flat stock-in, stock-out system.

7. Proposed Product Design

7.1 Main Purpose of the Product

The product should give KDPS a digital control layer for every store inventory event after inward acceptance.

It should help store staff, warehouse teams, operations heads, and head office teams answer the following questions at any moment:

- What stock is currently available in Store A?
 - What stock is physically inside the store but not available for normal sale?
 - What stock is on the way from Store A to Store B?
 - Which defects are pending brand replacement?
 - How much stock has been returned to the brand under seasonal terms?
 - How much stock was billed under EOSS but is still lying in the store?
 - How much of that billed-retained stock has later been sold physically?
 - What loss, if any, remains on residual inventory?
-

7.2 Integration with POS (Critical Layer)

This module must be tightly integrated with the **existing billing / Point of Sale (POS) system**. Without this integration, inventory accuracy will degrade immediately.

A. Standard Customer Sales (POS-driven)

- Every sale executed at POS must:
 - automatically deduct inventory from “**Available for Sale**” stock
 - update store inventory in real time
 - No manual stock deduction should be allowed for sales
 - POS becomes the **single source of truth for customer-driven inventory reduction**
-

B. EOSS / Internal Billing (Special Case)

This is where the system must behave differently.

In EOSS scenarios:

- Stock is **commercially billed** (treated as sold for brand settlement)
- But it **physically remains inside the store**

This creates a dual reality:

- Commercial ownership changes
- Physical stock remains

Required System Behavior

The POS or billing interface must support a **special billing mode**, such as:

- “Billed to Brand / Internal Billing”
- “EOSS Billing”
- “Self-Billed Inventory”

When this mode is used:

- Inventory should:
 - be removed from **Available for Sale**
 - be moved to a separate bucket:
→ **“Billed but Retained Inventory”**
- The system must capture an **identifier** for such billing:
 - could be:
 - a predefined internal customer (e.g., “EOSS Settlement Account”)
 - a brand-linked identifier
 - or a system-defined billing entity

This identifier is critical because it allows the system to distinguish:

Scenario	Meaning
Normal POS sale	stock leaves store physically
EOSS billing	stock is billed but remains in store

C. Tracking Billed-but-Retained Inventory

The system must maintain:

- count of such inventory per SKU
- store-level visibility
- linkage to:
 - billing event
 - pricing / discount applied

Later, when this stock is actually sold to a customer:

- it should be sold **without affecting brand liability again**
- system must:
 - reduce from “Billed but Retained” bucket
 - not double-count revenue or margin

D. Why This Integration Matters

Without POS integration:

- sales will not reflect in real time
- EOSS billing will get mixed with actual sales
- inventory numbers will become unreliable
- financial settlement with brands will be incorrect

This is not an optional feature.

It is a **foundational dependency**.

Final Positioning of the Module

This module is not just a store inventory tracker.

It is:

A state-driven inventory control system that connects physical stock, commercial events, and POS-driven reality into a single, accurate ledger

It ensures that:

- what is sold
- what is transferred
- what is returned
- what is billed but retained

are all captured distinctly, accurately, and in real time.

Module Name

Workforce Attendance, Presence & Payroll Control System (WAPCS)

1. Goal and Objective of the Module

This module is meant to solve a basic but important operational problem inside KDPS's store network:

Who actually came to work, when did they arrive, when did they leave, and how should salary and incentives be calculated based on reliable attendance data?

Right now, this truth does not exist in a structured way. The current practice of employees sending a photo from outside the store may give some indication of presence, but it does not create a reliable attendance system. It does not establish exact check-in time, exact check-out time, actual time spent inside the store, or whether the person remained available during working hours. It is also easy enough to manipulate that it cannot serve as a strong basis for payroll discipline.

So the objective of this module is not merely to “mark attendance.” Its purpose is to create a **trusted employee presence and attendance layer** that can later support:

- payroll processing
- incentive calculation
- late coming tracking
- absenteeism tracking
- shift discipline
- store staffing visibility
- auditability in case of disputes

This module also becomes the base for HR record management because employee documents such as PAN and Aadhaar need to be stored and associated with the employee profile.

Core objective

Create a mobile-first attendance and payroll support system that:

- reliably verifies store staff presence
- records clock-in and clock-out times
- minimizes fake attendance behavior
- creates a clean attendance ledger per employee
- supports employee profile and document management

- feeds attendance and incentive data into payroll logic

What success looks like

A successful version of this module would mean:

- No dependence on WhatsApp photos for attendance
 - Every employee has a unique, verified attendance identity
 - Store-level attendance can be seen in real time
 - HR can see who is present, late, absent, or missed check-out
 - Salary and incentives can be calculated from structured records
 - Employees can upload required documents digitally
 - Attendance disputes can be investigated with system logs rather than guesswork
-

2. What This Module Covers

This module covers four tightly connected areas:

A. Attendance capture

How the system knows an employee has arrived and left.

B. Presence verification

How the system reduces fake or proxy attendance.

C. Employee master and document management

How employee identity and statutory documents are stored.

D. Payroll input generation

How attendance records and incentive data are prepared for salary calculation.

This module does **not** need to become a full HRMS on day one. That would be a common mistake. If you try to build leave management, recruiting, appraisal, training, and full payroll tax logic all at once, the module will become bloated and slow to adopt.

The right first scope is narrower:

- identify employee
- mark attendance
- track timing

- store employee documents
 - produce structured payroll inputs
-

3. Current Business Reality

Today, there is no formal attendance control system across stores.

The current practice is roughly this:

- employee reaches store
- employee sends a photo showing presence in front of the store
- management informally assumes attendance

This method has obvious weaknesses:

- it proves little beyond a momentary photo
- it does not reliably prove start time
- it does not prove end time
- it does not prove continuous presence
- it can be faked
- it cannot directly support payroll logic

There is also no formal punch machine, and there is no desire to introduce a traditional hardware punch-in system. That is a reasonable instinct, because old punch systems often create their own problems:

- hardware procurement
- breakdowns
- installation effort
- store-level maintenance
- dependency on vendor support

At the same time, avoiding punch machines does not remove the need for reliable attendance. It simply means the system has to be designed around the device employees already use most naturally: the smartphone.

4. The Business Problem This Module Must Solve

The problem is bigger than “attendance.”

KDPS needs answers to several questions that today cannot be answered reliably:

- Which staff members are present in the store right now?
- Who came late?
- Who left early?
- Who never checked in?
- Who checked in but forgot to check out?
- Which store is understaffed today?
- How many working hours did a person actually complete?
- On what basis should salary deductions or incentives be calculated?

There is also a second layer to this problem:

Attendance must be easy enough to use daily, but strong enough that people cannot casually game it.

That tension matters. Many attendance systems fail because they are designed only for control, not for adoption. Others fail because they are too loose and become symbolic rather than reliable.

So the design has to balance:

- ease of daily use
 - fraud resistance
 - low hardware dependency
 - simple rollout across stores
-

5. Options Under Consideration

You've already described two possible approaches. They are not equally practical, and it is important to evaluate them properly before writing the product as if both are equally good.

Option 1: Store-Owned Android Device with Face-Based Check-In / Check-Out

How it would work

- Each store has one dedicated Android phone or tablet
- Device remains inside the store
- Employee arrives
- Opens attendance app
- System verifies face against stored profile
- Attendance is marked as clock-in
- At end of shift, same process used for clock-out

What this solves well

- Standardized hardware environment
- Lower dependence on employee phone quality
- Easier to control app version and device settings
- Presence check happens on a device physically kept at store
- Much easier to supervise operationally
- Simpler to train staff on one fixed process

Risks / limitations

- Someone could still try proxy attendance using face spoofing if liveness detection is weak
- Device could be unavailable, uncharged, or misused
- If multiple staff arrive together, queuing might happen
- Missed checkout needs fallback logic
- Device theft or damage needs handling

Implementation complexity

Moderate. This is the more practical and stable approach.

Option 2: Employee-Owned Mobile App + Wi-Fi / IP-Based Presence Tracking

How it would work

- Employee has app on personal phone
- App detects store Wi-Fi or known IP/network presence
- System infers entry time, exit time, and possible out-of-range behavior

What this tries to solve

- No dedicated hardware at store
- Potentially richer movement and presence insight
- Passive attendance detection could reduce friction

Why this is harder than it sounds

This approach looks elegant, but operationally it is much weaker and more complex.

Problems include:

- employee may keep Wi-Fi off
- employee phone may have battery issues

- Wi-Fi may fluctuate
- multi-floor store layouts may distort signal behavior
- short disconnects may create false exits
- people may leave phone behind
- some Android manufacturers restrict background processes
- continuous monitoring creates more privacy and support complexity

Implementation complexity

High. It is harder to build, harder to trust, and harder to support.

6. Which Option is Easier and Better to Implement First

The easier and more reliable first implementation is:

Recommended Approach: Store-Owned Android Device + Face Verification

This is the better first version for KDPS.

Why this is the right choice

1. It is operationally simpler

A fixed device at store is easier to control than many personal phones with different settings, permissions, and hardware behaviors.

2. It is easier to train

Employees learn one process:

- come in
- check in using store device
- leave
- check out using store device

3. It reduces environmental uncertainty

You do not need to depend on:

- Wi-Fi strength behavior
- employee network settings
- background tracking permissions

4. It creates a stronger compliance ritual

When attendance is tied to a visible device at the store, adoption becomes clearer and easier to monitor.

5. It is more realistic for Phase 1

You can get a working system faster with fewer edge-case failures.

7. Current Workflow (As-Is)

Attendance today

- Employee arrives at store
- Sends photo as proof of presence
- No formal check-in or check-out timestamp
- No structured attendance register

Employee records today

- Documents may be scattered or maintained manually
- No integrated employee profile system

Payroll today

- Salary and incentives are calculated without a strong attendance source of truth
- Potential for disputes, inconsistency, and missed deductions or credits

Where the current system breaks

A. No reliable attendance truth

A photo is not a proper check-in record.

B. No exit record

Even if arrival is loosely indicated, departure is not captured properly.

C. No continuous auditability

There is no structured event log.

D. Weak payroll linkage

Attendance data is not clean enough to drive salary logic confidently.

E. Easy manipulation

A photo can be sent without strong assurance of actual presence.

8. System Gaps and Improvement Areas

A. No verified attendance identity

Need a strong link between person and attendance event.

B. No attendance event ledger

Need structured records:

- check-in time
- check-out time
- missed punch
- late mark
- manual override

C. No employee master record

Need employee profiles with:

- name
- employee ID
- store assignment
- role
- PAN
- Aadhaar
- salary structure
- incentive plan

D. No exception handling framework

Need logic for:

- missed check-out
- missed check-in
- device offline
- manual correction request

E. No payroll-ready outputs

Need attendance summaries and payable days calculation.

9. Key Improvement Principle

This module should not be treated as a digital register.

It should be built as a **verified attendance and payroll input system**.

That means every attendance record should be treated as an event with:

- who
- where
- when
- how verified
- whether accepted automatically or overridden manually

This matters because later payroll disputes are not solved by totals. They are solved by event history.

10. Proposed Product Design (To-Be)

10.1 System Overview

The system should include:

- a store-facing attendance device workflow
- a web-based HR/admin dashboard
- an employee profile and document layer
- an attendance processing engine
- a payroll input generation layer

The Android attendance device is the operational touchpoint. The web dashboard is the administrative control point.

10.2 User Interfaces

A. Store Attendance App (Dedicated Android Device)

Used for:

- check-in
- check-out
- facial verification
- possibly viewing today's attendance status for current shift

Core features

- employee search or employee code entry
 - face verification
 - attendance success/failure screen
 - timestamp capture
 - offline fallback queue if internet is unstable
 - store/device identity tagging
-

B. Employee Self-Service Interface

Can be mobile web or app-lite depending on rollout choice.

Used for:

- viewing attendance history
- uploading documents
- checking profile details
- possibly raising correction request later

This is useful, but it does not need to be overbuilt in Phase 1.

C. HR / Admin Web Dashboard

Used by:

- HR
- store operations manager
- payroll/admin team

Core features

- employee profile creation
- document upload and approval
- store assignment
- attendance history
- late / absent reports

- missed checkout review
 - payroll input export
 - incentive configuration
-

10.3 System Components

A. Face Verification Engine

- stores face templates during onboarding
- validates check-in/check-out match
- should ideally include liveness detection to reduce spoofing risk

B. Attendance Event Engine

Captures:

- check-in
- check-out
- missed event
- manual correction
- override approval

C. Device & Store Binding Layer

- every attendance device mapped to store
- attendance event automatically tagged to that store

D. Employee Master System

Stores:

- employee details
- documents
- salary structure
- incentive plan
- assigned store

E. Attendance Rules Engine

Applies business rules such as:

- shift timing
- grace period
- half-day rules

- missed punch handling
- late arrival logic

F. Payroll Layer

Generates structured outputs for:

- days present
- absences
- late marks
- payable days
- attendance-linked incentive calculations

This module should feed payroll, not replace the entire payroll system immediately.

The system should calculate and expose:

- total present days
- absent days
- half days
- late marks
- missed punch events
- payable working days
- incentive-linked attendance metrics

You also mentioned salary and incentive calculation. That should be supported, but carefully.

10.4 Core Workflow

Flow 1: Employee Onboarding

1. HR creates employee profile
2. Employee details added
3. PAN and Aadhaar uploaded
4. Face enrollment completed
5. Store assigned
6. Salary and incentive rules associated

Flow 2: Check-In

1. Employee arrives at store
2. Opens attendance app on store device
3. Face is scanned
4. System verifies identity

5. Check-in time recorded
6. Attendance status marked for the day

Flow 3: Check-Out

1. Employee leaves store
2. Repeats face verification
3. Check-out time recorded
4. Work duration calculated

Flow 4: Exception Handling

If employee:

- forgets checkout
- fails verification
- device is offline
- attendance needs correction

Then:

- correction request or manual admin action is created
 - approval trail stored
-

10.5 Future Workflow (End-to-End)

Employee Onboarded



Documents Verified



Face Enrolled



Store Device Check-In



Attendance Event Recorded



Store Device Check-Out



Daily Attendance Finalized



Attendance Rules Applied



Payroll Input Generated

11. Key Edge Cases the Product Must Handle

A. Missed checkout

Employee checks in but does not check out.

System should:

- flag record as incomplete
- send to manager/HR review
- apply policy-based resolution

B. Face mismatch

Wrong person or failed scan.

System should:

- deny automatic check-in
- allow retry
- escalate if repeated

C. Device offline

Attendance should not disappear because internet fails.

Need:

- local timestamp queue
- sync when online

D. Employee moved to different store

Attendance authorization should follow current store assignment rules.

E. Store opens late or unusual shift

Need override capability without destroying audit trail.

F. Proxy attempts

Need liveness checks and audit logs.

12. Final Positioning

This module is not just an attendance tracker.

It is:

A verified workforce presence and payroll control layer that gives KDPS reliable staff attendance truth without using legacy punch machines

It creates the operational foundation for:

- staffing visibility
- discipline
- payroll confidence
- incentive governance